



RISC-V Semihosting

Version 0.4, 12th June 2024: This document is under development. Expect potential changes.

Table of Contents

Preamble	1
Copyright and license information	2
Contributors	3
1. Introduction	4
2. RISC-V Semihosting Binary Interface	5
2.1. Semihosting Trap Instruction Sequence	5
2.2. Semihosting Parameters	5
Bibliography	7

Preamble



This document is in the *Development state*

Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.

Copyright and license information

This specification is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC-BY-SA-4.0). The full license text is available at creativecommons.org/licenses/by-sa/4.0/.

Copyright 2024 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Krste Asanovic <krste@sifive.com>
- Palmer Dabbelt <palmer@dabbelt.com>
- Liviu Ionescu <ilg@livius.net>
- Keith Packard <keith.packard@sifive.com>
- Megan Wachs <megan@sifive.com>
- Anup Patel <apatel@ventanamicro.com>

Chapter 1. Introduction

Semihosting is a technique where an application running in a debug or simulation environment can access elements of the system hosting the debugger or simulator including console, file system, time and other functions. This allows for diagnostics, interaction and measurement of a target system without requiring significant infrastructure to exist in that target environment.

The RISC-V semihosting specification adapts the design of publicly available and open source semihosting specification [1] to minimize the development effort. Only the mechanism of invoke a semihosting service (aka semihosting binary interface) is architecture specific whereas all defined semihosting services [1] are architecture independent and resemble system calls of an unix-style operating system. The [Figure 1](#) below shows an architecture independent high-level view of semihosting usage.

This specification only defines the semihosting binary interface for RISC-V platforms and all other aspects of semihosting are defined by the open source semihosting specification [1].

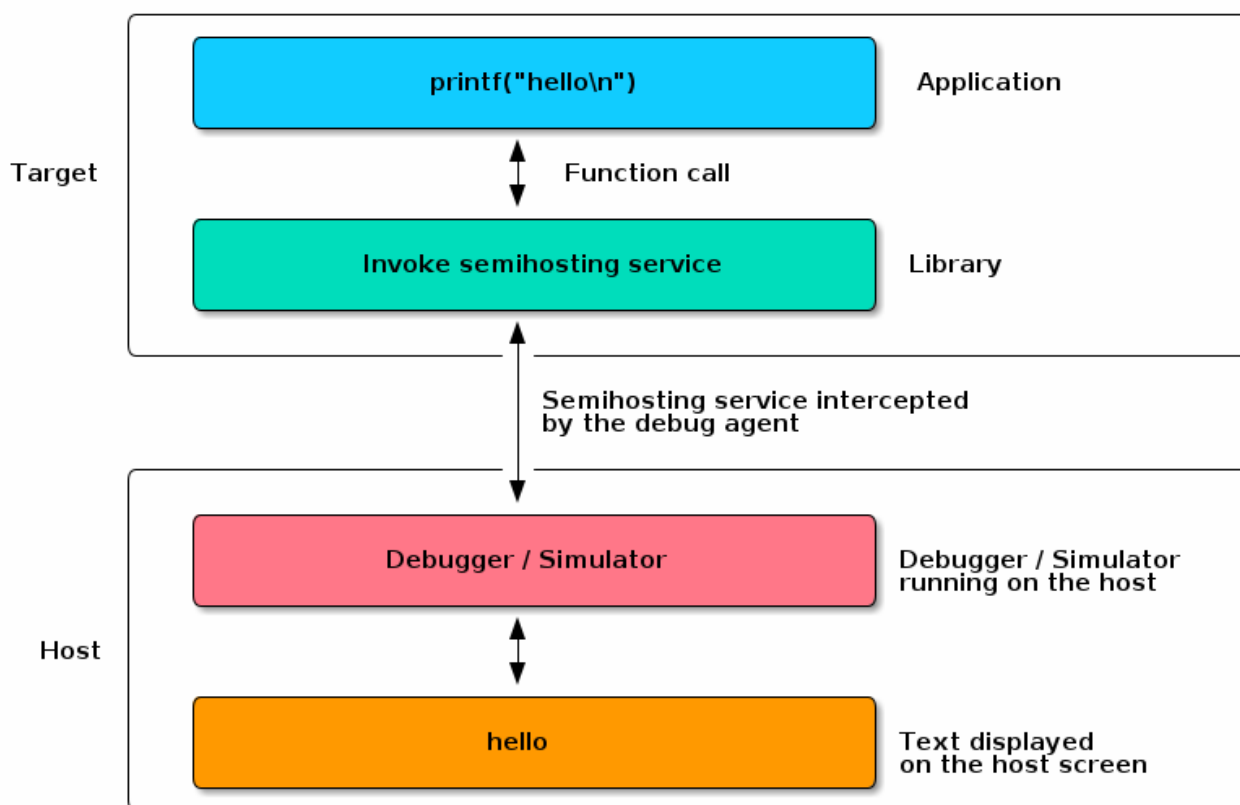


Figure 1. Generic Semihosting Usage Flow

Chapter 2. RISC-V Semihosting Binary Interface

The RISC-V semihosting binary interface consist of a trap instruction sequence and mechanism to pass parameters which are defined by the following sub-sections.

2.1. Semihosting Trap Instruction Sequence

Semihosting operations are requested using a sequence of instructions including **ebreak**. Because the RISC-V base ISA does not provide more than one **ebreak** instruction, RISC-V semihosting uses a special sequence of instructions to distinguish a semihosting **ebreak** from a debugger inserted **ebreak**. The [Listing 1](#) shows the instruction sequence used to invoke a Semihosting operation.

```
slli x0, x0, 0x1f    # 0x01f01013    Entry NOP
ebreak              # 0x00100073    Break to debugger
srai x0, x0, 7       # 0x40705013    NOP encoding the semihosting call
number 7
```

Listing 1. RISC-V Semihosting Trap Sequence

These three instructions must be 32-bit-wide instructions, they may not be compressed 16-bit instructions. This same sequence is used on all RISC-V architectures. If paging is in use in the current mode, this sequence must not cross a page boundary as the semihosting system must be able to check for the semihosting sequence without needing data from potentially missing pages. The [Listing 2](#) shows how this can be done by placing the sequence in a separate function and aligning that to prevent that from spanning a page boundary.

```
.option norvc
.text
.balign 16
.global sys_semihost
.type sys_semihost @function
sys_semihost:
    slli zero, zero, 0x1f
    ebreak
    srai zero, zero, 0x7
    ret
```

Listing 2. RISC-V Semihosting Trap Function

2.2. Semihosting Parameters

The type of semihosting operation and it's parameter are specified using general purpose registers. The [Table 1](#) shows the specific registers that are used, and the size of the fields in the data block, which depend on whether the caller is 32-bit or 64-bit.

Table 1. RISC-V Registers and field size

	32-bit	64-bit

OPERATION NUMBER REGISTER	A0	A0
PARAMETER REGISTER	A1	A1
RETURN REGISTER	A0	A0
Data block field size	32 bits	64 bits

Bibliography

[1] “Semihosting for AArch32 and AArch64 2023Q3.” 2023, [Online]. Available: github.com/ARM-software/abi-aa/releases/download/2023Q3/semihosting.pdf.